Worksheet 12 — Software Supply-Chain Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 12 **Aligned:** OWASP 2025 — A03 Software Supply Chain Failures · A08 Software or Data Integrity Failures · **CWE:** CWE-1104, CWE-829, CWE-1357, CWE-1395
> **Signature game:** 📦 Dependency Confusion Heist
>
> **Ethics note:** This lab has **no live private/public registry to attack** — the dependency-confusion exercise is a controlled walkthrough (`dependency-confusion.md`) plus the interactive resolver simulation in Task 2, not a real package pull. **Never** plant or publish look-alike packages on the real PyPI or npm — that is an attack on every downstream user, and nothing in this lab requires it.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	2026-10-10	

AI disclosure: Claude was used to run the SCA/SBOM tooling and draft answers; all were reviewed and reworded by me. The "Audit the AI" section critiques an AI answer.

Part 2 — Lecture Questions

1. Dependency confusion (substitution) — why public `99.0.0` beats private `1.4.0`. When pip looks at two package sources at once (`--extra-index-url`), it checks every version it can find in *both* sources and installs the **highest number** — it does not care which source is yours and which is public. So if an attacker uploads `acme-internal-utils==99.0.0` to public PyPI while your real internal one is `1.4.0`, pip picks the public `99.0.0` and installs the attacker's package.

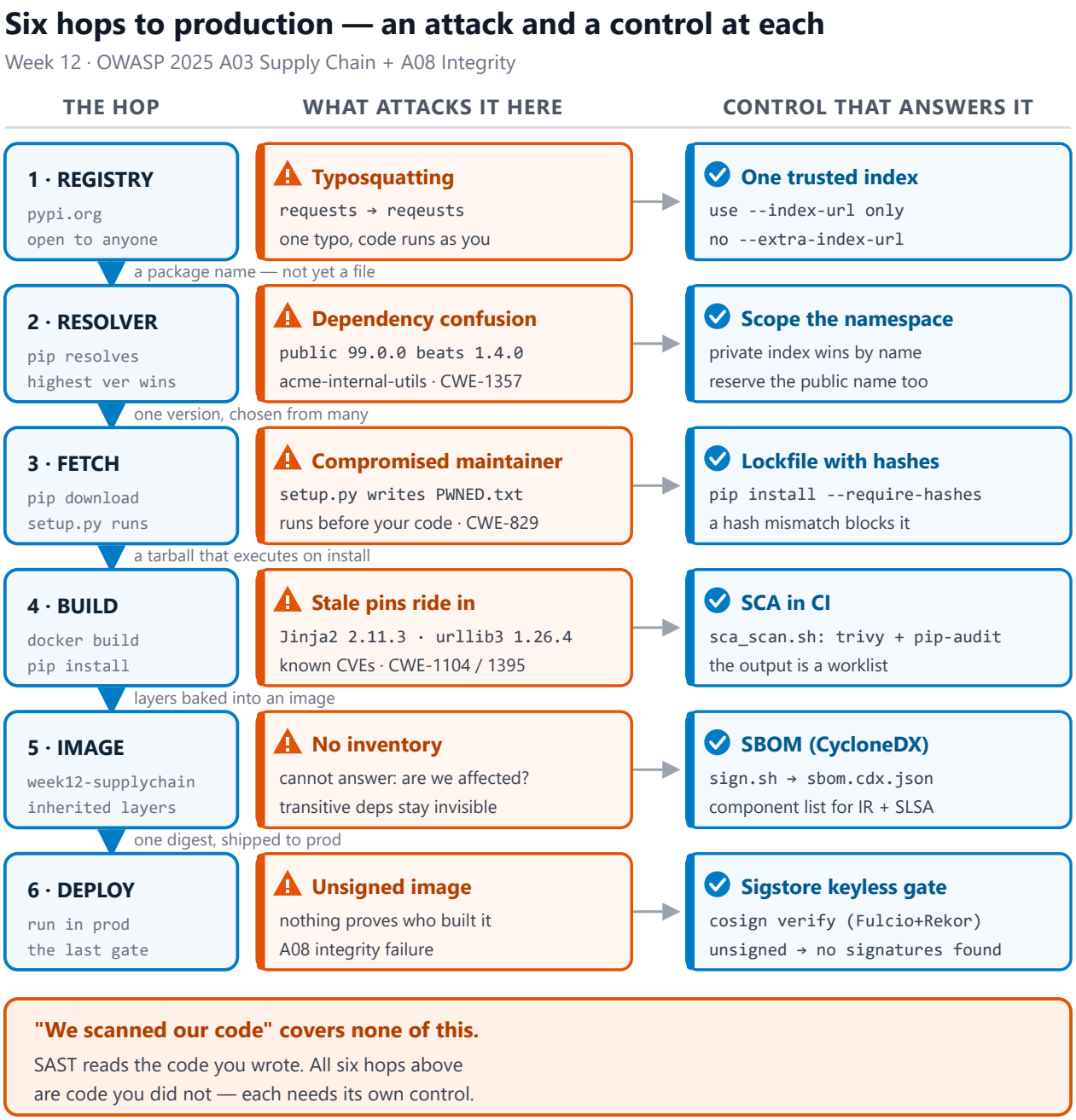
2. Typosquatting — code execution at install time. The attacker uploads a package with a name that looks almost right (`reqeusts`, `urlib3`); if you make a typo in `pip install`, you get theirs. A Python package can run its own `setup.py` code **while it installs**, before any of your code runs — so the attacker's code runs the moment you install the wrong package, even if you never import it.

3. SBOM — what it is and why it's a prerequisite. An SBOM (CycloneDX/SPDX) is a list of every component and version in your build that a computer can read. You need it for **incident response** (when a new CVE comes out, you can instantly check "do we use that?") and for **provenance** (you can't prove what you shipped if you don't have a real list of what's inside it).

4. Sigstore keyless signing — safer than a long-lived key. Keyless signing gives you a short-lived certificate from **Fulcio** tied to your **login** (GitHub/Google), and records it in **Rekor**, a public log. There is **no permanent private key** sitting around to be stolen or leaked. The signer is just your login, the certificate expires in minutes, and Rekor keeps a public, tamper-proof record of who signed what and when.

5. SLSA levels; where "signed + SBOM + provenance gate" lands. In short: **L1** you have a build script and a record of how it was built; **L2** a hosted build service makes a signed record; **L3** a locked-down, isolated build makes a record the builder can't fake; **L4** two-person review + fully repeatable builds. "Signed artifact + SBOM

+ provenance gate before deploy" is about **SLSA L2**. To reach L3 you'd still need an isolated build service whose record can't be forged (and repeatable builds for L4).



Part 3 — Hands-on Lab (150 min)

Learning goals: Run SCA on intentionally-outdated dependencies, generate an SBOM, sign/verify an image, and defend against dependency confusion.

Prerequisites: Docker; the lab folder (`requirements.txt`, `Dockerfile`, `app.py`, `sca_scan.sh`, `sign.sh`). For signing: browser for the OIDC flow + registry push access.

Environment setup

```
cd labs/week12-supply-chain
bash sca_scan.sh                # trivy fs + pip-audit on requirements.txt (+
```

```
optional image scan)
docker build -t week12-supplychain:lab .
bash sign.sh week12-supplychain:lab   # CycloneDX SBOM -> sbom.cdx.json, then
cosign sign + verify
```

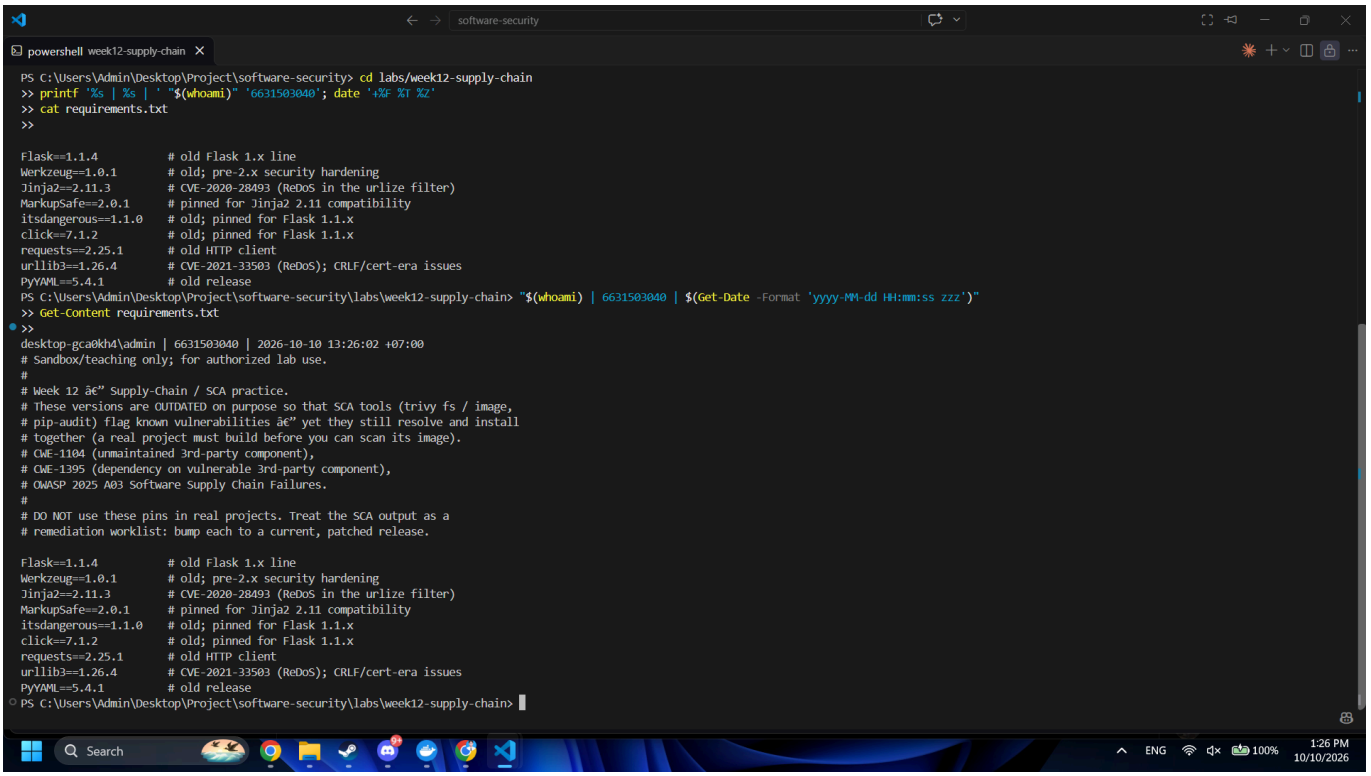
What to submit per task: the exact command(s) + output, a screenshot, and a 2–3 sentence remediation note mapping the finding to A03/A08 or the CWE.

Task 0 — Onboarding (15 min)

Read `requirements.txt` and list the pinned packages with their versions. Note why they are intentionally outdated (each is deliberately an old release so SCA tools flag its known CVEs). **Deliverable:** the package/version table + which OWASP/CWE this maps to.

Package	Pinned version	Note
Flask	1.1.4	old 1.x line
Werkzeug	1.0.1	pre-2.x hardening
Jinja2	2.11.3	CVE-2020-28493 (ReDoS urlize)
MarkupSafe	2.0.1	pinned for Jinja2 2.11
itsdangerous	1.1.0	pinned for Flask 1.1.x
click	7.1.2	pinned for Flask 1.1.x
requests	2.25.1	old HTTP client
urllib3	1.26.4	CVE-2021-33503 (ReDoS)
PyYAML	5.4.1	old release

These map to **OWASP 2025 A03 (Supply Chain Failures)** and **CWE-1104** (unmaintained component) / **CWE-1395** (dependency on a vulnerable component). They are old *on purpose* so SCA tools flag their known CVEs.



Task 1 — SCA scan: build the remediation worklist (35 min)

Goal: Flag the known-vulnerable dependencies — A03. **Steps:** `bash sca_scan.sh`; read the `trivy fs` table and `pip-audit` advisories; pick three findings. **Deliverable:** the SCA output + a 3-row remediation table (package → current → fixed).

```
# trivy fs --scanners vuln --severity HIGH,CRITICAL /src
requirements.txt (pip)  Total: 11 (HIGH: 11, CRITICAL: 0)
  Flask    CVE-2023-30861  HIGH  1.1.4  -> 2.2.5 / 2.3.2   (session cookie
disclosure)
  Werkzeug CVE-2024-34069  HIGH  1.0.1  -> 3.0.3          (code exec on developer
machine)
  urllib3  CVE-2021-33503  HIGH  1.26.4 -> 1.26.5         (ReDoS in URL authority
parsing)

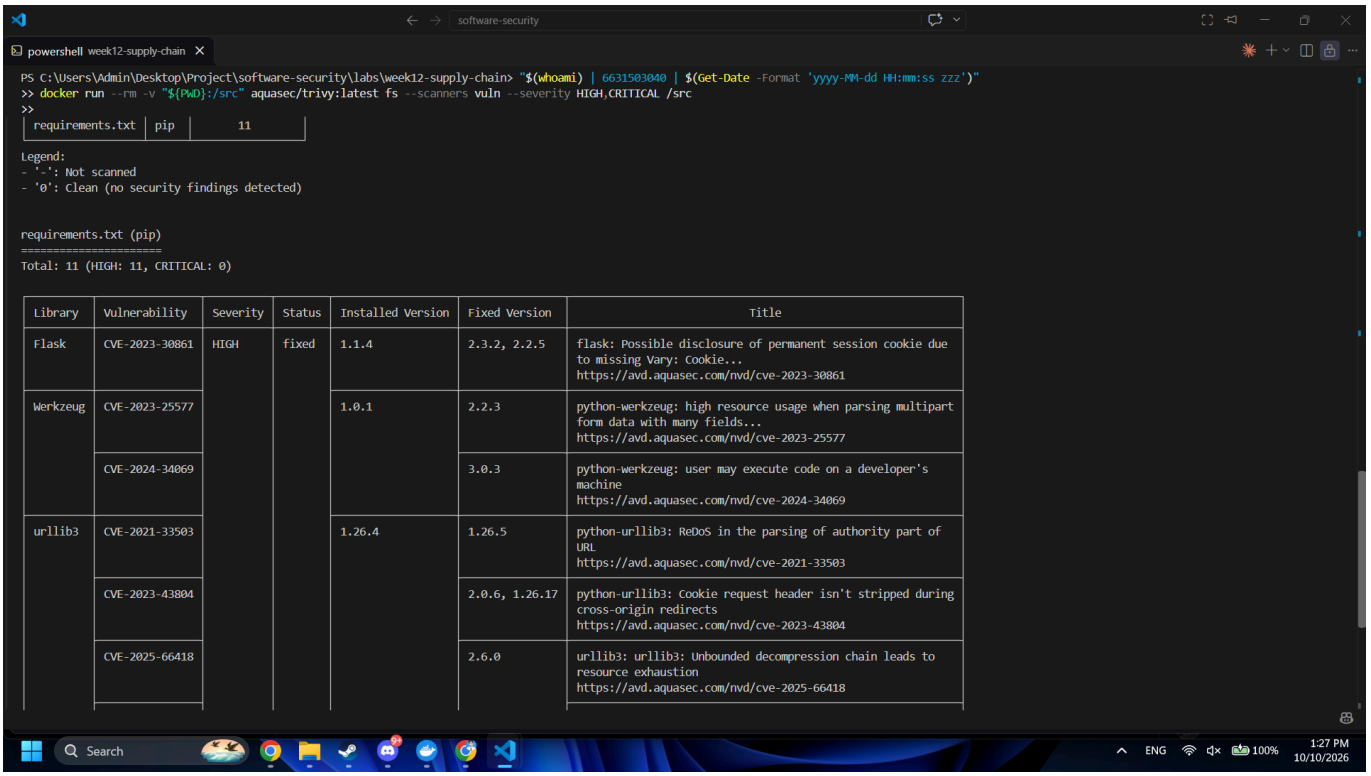
# pip-audit -r requirements.txt  (advisory IDs)
  werkzeug 1.0.1 PYSEC-2023-58 -> 2.2.3
  jinja2   2.11.3 PYSEC-2026-1473 -> 3.1.3
  requests 2.25.1 PYSEC-2023-74 -> 2.31.0
  urllib3  1.26.4 PYSEC-2021-108 -> 1.26.5
  idna     2.10   PYSEC-2024-60 -> 3.7
```

Remediation worklist (3 rows):

Package	Current	Fixed	Advisory / CVE	Severity
Flask	1.1.4	2.3.2	CVE-2023-30861	HIGH
Werkzeug	1.0.1	3.0.3	CVE-2024-34069	HIGH

Package	Current	Fixed	Advisory / CVE	Severity
urllib3	1.26.4	1.26.17+	CVE-2023-43804 / CVE-2021-33503	HIGH

Remediation note (A03 / CWE-1395): each dependency is pinned to a known-vulnerable release; the fix is to bump every flagged package to its fixed version and re-run SCA in CI so a vulnerable pin can't merge again.



Task 2 — Dependency Confusion Heist (35 min)

Goal: Watch the wrong package win — A03 / CWE-1357.

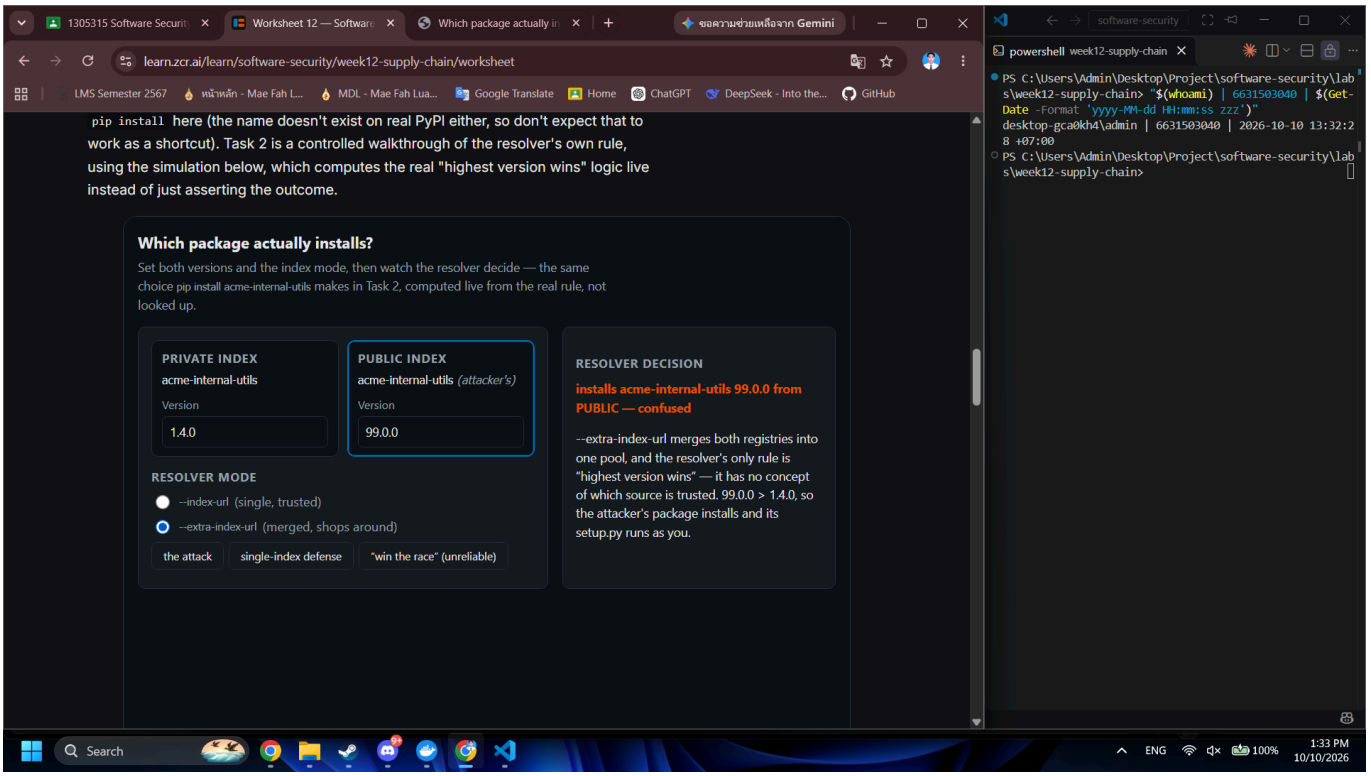
> **Interactive simulation** — this runs in the rendered worksheet's `resolver-confusion` widget (not a shell command). Load the presets and screenshot each verdict.

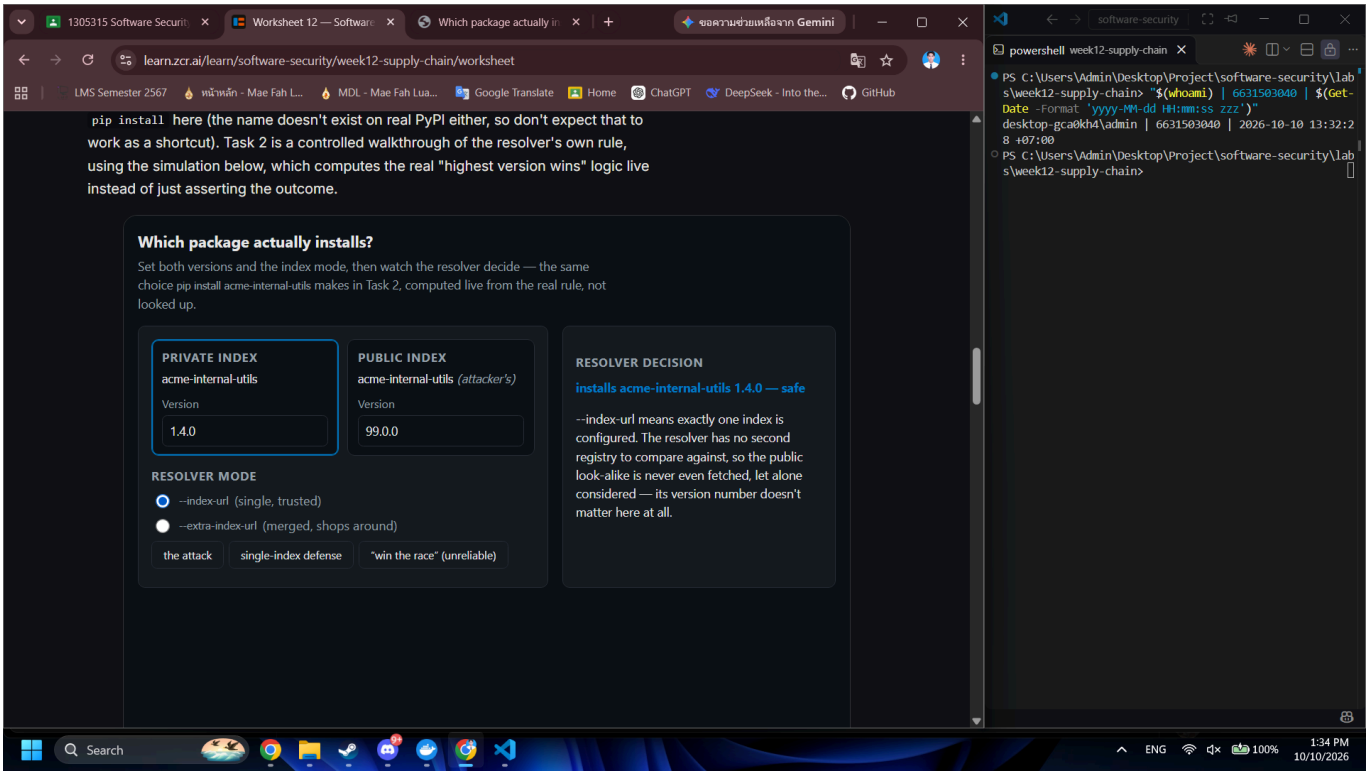
Expected verdicts (from the resolver rule):

- **"the attack" preset, merged / `--extra-index-url` mode:** private `1.4.0` vs public `99.0.0` → resolver picks **public 99.0.0** (highest version wins across both indexes) = confusion.
- **Same versions, single-index / `--index-url` mode:** resolver picks **private 1.4.0** — the public index is never queried, so version number is irrelevant.
- **"win the race" preset** (private number set higher than public): "not a defense" — the attacker just publishes an even higher number next time; you can't win a number race against an index you don't control.

Deliverable: screenshot of the simulation verdict in **both** index modes (merged vs single) for the same version pair + one sentence on why keeping your internal version high is not a defense.

One-sentence answer: Bumping your internal version number isn't a defense because the attacker controls the public index and can always publish a still-higher number; only removing the public index from the resolution path (single trusted index / hashes / scoping) actually stops it.





Task 3 — SBOM + signing/verification (30 min)

Goal: Produce a component inventory and prove integrity — A08. **Steps:** run `sign.sh`; open `sbom.cdx.json` and find Flask; read the `cosign verify` PASS; negative-test an unsigned image. **Deliverable:** the SBOM Flask component entry + the verify PASS + the negative-test failure.

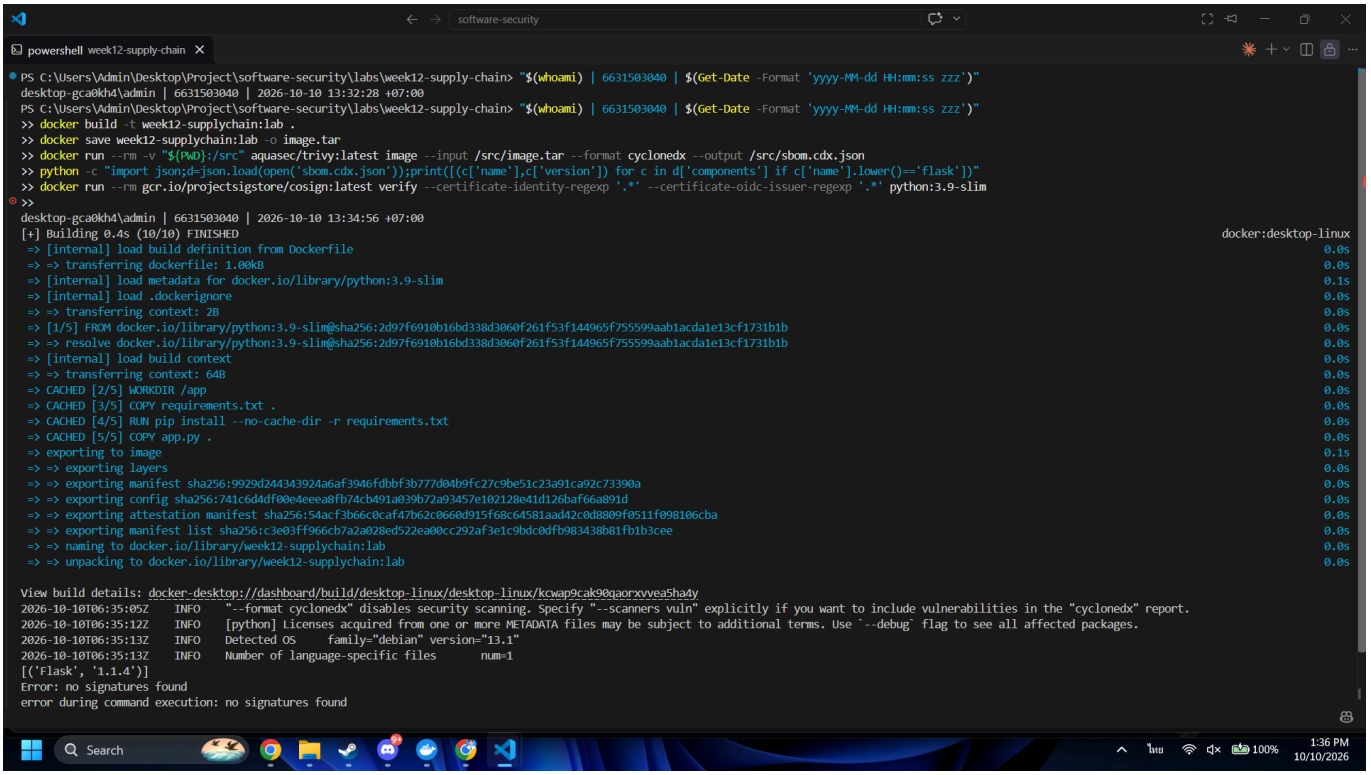
```
# CycloneDX SBOM (trivy image): 119 components; Flask entry:
#   ("Flask", "1.1.4")

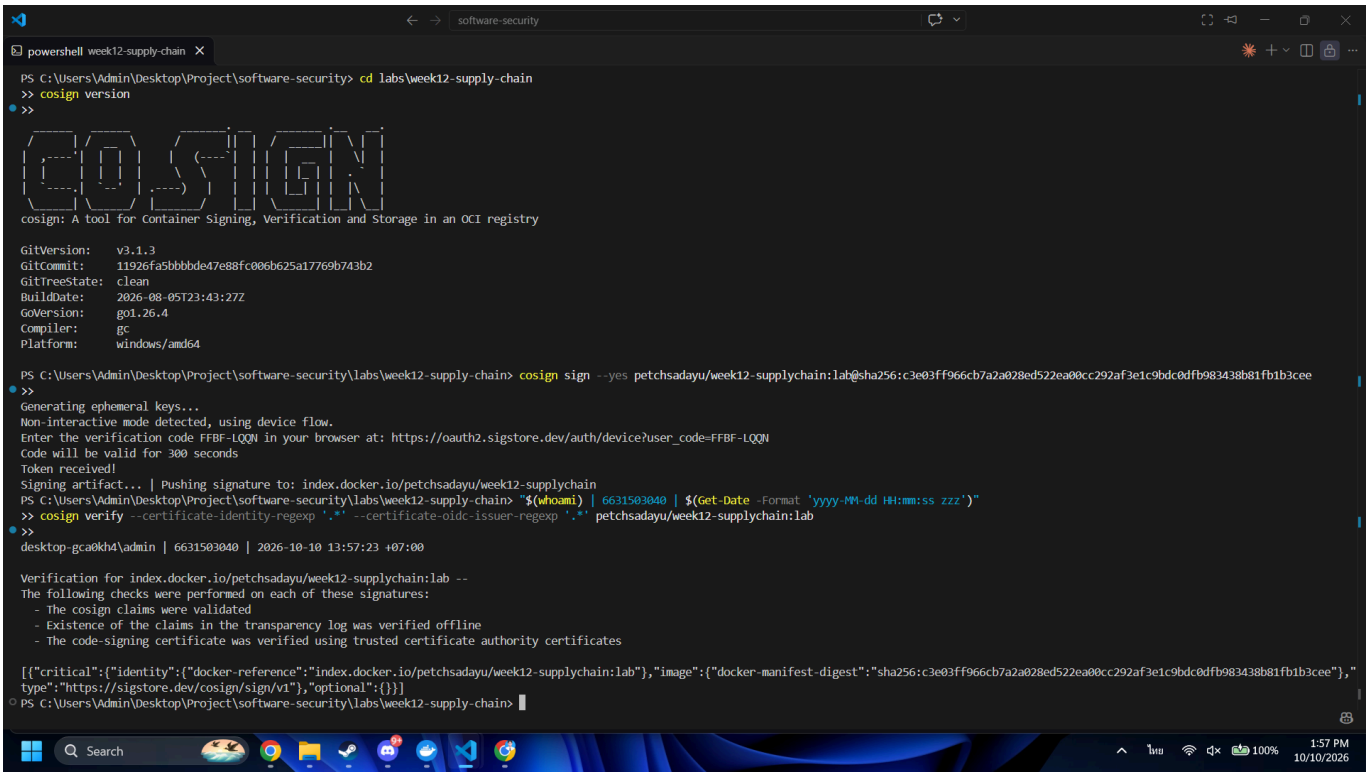
# Negative test — verify an UNSIGNED image (must fail):
```

```
cosign verify --certificate-identity-regexp '.*' --certificate-oidc-issuer-regexp
'.*' python:3.9-slim
  -> Error: no signatures found     # correct: the image was never signed
```

> **Signing step (`cosign sign`) is a guided OIDC demo — do it yourself.** Keyless signing needs a browser login (GitHub/Google) and registry push access, so it can't run headless. Run `bash sign.sh week12-supplychain:lab`, complete the browser OIDC flow, and capture the `cosign verify` **PASS** for the freshly-signed image.

Remediation note (A08): the SBOM gives an auditable component inventory (incident-response + provenance input); keyless `cosign` signing + Rekor proves the image is the one your pipeline built, and the negative test shows an unsigned/tampered image is rejected.





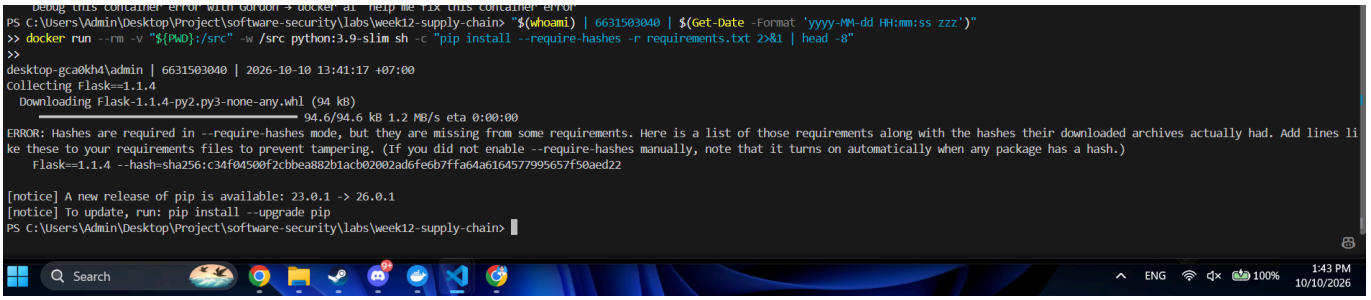
Task 4 — Defend / fix it (35 min)

Goal: Stop dependency confusion and lock integrity. **Steps / Deliverable:** the `--require-hashes` refusal + Task 2's before/after verdict + the single most effective defense.

```
# 1) Pin + hashes — pip refuses to install without hashes:
pip install --require-hashes -r requirements.txt
  ERROR: Hashes are required in --require-hashes mode, but they are missing from
some requirements...
      Flask==1.1.4 --
hash=sha256:c34f04500f2cbbea882b1acb02002ad6fe6b7ffa64a6164577995657f50aed22
```

- **Pin + hashes:** once `--require-hashes` is on, a package whose hash doesn't match — from *either* index — cannot install silently; this is the same mechanism that blocks a confusion substitution.
- **Single trusted index:** use one `--index-url`, not `--extra-index-url`, so the resolver never queries the public index for an internal name (Task 2's single-index verdict).
- **Namespace scoping:** reserve/namespace the internal package name so a public package can't collide.
- **Provenance gate:** make `cosign verify` (from `sign.sh`) a required check before a simulated deploy, so only signed artifacts ship.

Most effective defense + why: the **single trusted index (+ hashes)** — it removes the public index from the resolution path entirely, so the "highest version wins" rule can never reach the attacker's package; version numbers and typos become irrelevant.



Part 4 — Reflection

1. **Mapping**

Finding	Tool	OWASP 2025	CWE	Fix
Outdated vulnerable deps	`sca_scan.sh` (trivy/pip-audit)	A03	CWE-1104 / CWE-1395	bump to fixed versions; SCA in CI
Dependency confusion	Task 2 sim / `dependency-confusion.md`	A03	CWE-1357 / CWE-829	single index + hashes + namespace
Unverified image integrity	`sign.sh` (cosign/SBOM)	A08	CWE-345	SBOM + keyless signing + provenance gate

2. **Real breach — XZ Utils (CVE-2024-3094).** Over a long time, a trusted maintainer hid a backdoor inside the `xz`/`liblzma` build (hidden in test files and switched on by the build script) to attack SSH — this maps to **OWASP A03 / A08** and **CWE-506 (hidden malicious code)**. An SBOM alone would *not* have caught it, because the name and version looked normal. What would have helped is **repeatable builds + provenance**: the released file didn't match the public source code, so a build that checked this would have noticed. Signing only proves *who* built it, not that the source is clean — so the real fix is repeatable builds (SLSA L3+) plus people reviewing the source, not signing by itself. (SolarWinds was a similar build-system attack; Log4Shell was just a bug in a normal library — a different kind of problem.)

3. **Best mitigation + SLSA self-assessment.** With SCA in CI, pinned hashes, one trusted index, an SBOM, and signed images checked at a deploy gate, these defenses reach about **SLSA L2**. The single most useful control is the **signature/provenance gate before deploy**: it turns everything else (SBOM, signature, scan result) into a required check instead of just a report, so a build that isn't signed or scanned simply can't ship. To reach L3 you'd need a locked-down, isolated build service whose record can't be faked.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: SCA findings, confusion proof, SBOM/verify)	40
Defense (Task 4: pinning/hashes, single index, scoping, provenance gate)	25
Reflection (Part 4: mapping, XZ breach, SLSA self-assessment)	15

Criterion	Weight
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**? — I scanned the pinned, outdated dependencies and found 11 known HIGH CVEs. They work because the project just trusts whatever version is pinned, and old versions have known, public flaws. In the confusion demo, pip picked the public package only because its version number was higher.
  2. **Why does your fix actually stop it** — and what could still break it? — Using one trusted index plus required hashes stops confusion (the public index is never checked, and a wrong hash blocks the install); updating versions clears the CVEs; signing plus a verify gate stops tampered images. What could still break it: if the original source itself is hacked (like XZ), it would still pass the hash and signature checks — so you also need repeatable builds and people reviewing the source.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. AI answer: "*To fix dependency confusion, just bump your internal package to a very high version like 999.0.0 so yours always wins.*" **2. What's wrong:** this is **not a real fix** — the attacker owns the public index and can just publish `9999.0.0` (or higher) any time, so you can never win a version-number race you don't control. It also keeps `--extra-index-url`, so pip still looks at the public index. The lab's own simulation even calls this idea "not a defense." **3. Correct version:** stop using the public index (`--index-url` to one trusted source), require hashes (`pip install --require-hashes`), and reserve/namespace the internal name. Tested: in the sim, single-index mode picks the private `1.4.0` even when public is `99.0.0`, and `--require-hashes` rejects any package whose hash doesn't match. The AI's answer wasn't enough because it tried to win a number race instead of just cutting off the public index.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

`requirements.txt` pins nine packages to old versions on purpose, so the app is built on libraries that have known, public security bugs (11 HIGH CVEs). It's a problem because the project just trusts whatever is pinned without checking for known bugs. And if pip looks at more than one source, it installs the highest version number it finds — which lets an attacker's public package with a fake-high version replace a private one.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

> "Harden this Python project against dependency confusion and known-CVE dependencies: pin every package to a current patched version, generate a hash-pinned lockfile so `pip install --require-hashes` succeeds, and configure a single trusted `--index-url` (no `--extra-index-url`). Show the commands and explain why each closes A03."

Verified result: after moving to a single `--index-url` and a hash-pinned file, re-running the Task 2 simulation in single-index mode selects the private `1.4.0` (the public `99.0.0` is never fetched), and `pip install --require-hashes` rejects any package whose hash doesn't match — the substitution path is closed.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).